6주차 과제

서술형 1번. 기능별 최적의 데이터베이스 설계

1) 관리자용 통계 및 추천 성능 분석 — 데이터 웨어하우스

운영팀과 데이터팀은 일 단위 추천 클릭률, 구매 전환율, 인기 스타일 변화, 사용자 세그먼트별 반응 등을 분석해야 한다. 이러한 분석성 질의는 운영 DB에 직접 수행하면 서비스 성능에 부담을 줄 수 있다. 따라서 로그와 주문 데이터를 주기적으로 적재해 분석하는 **데이터 웨어하우스형 저장소**를 별도로 두는 것이 좋다.

2) 사용자 활동 로그 및 클릭 이력 저장 — NoSQL

추천 시스템을 개선하려면 사용자가 어떤 상품을 클릭했는지, 얼마나 오래 봤는지, 찜했는지, 장바구니에 넣었는지와 같은 **행동 로그**가 지속적으로 쌓여야 한다. 이런 데이터는 양이 많고 구조가 자주 바뀔 수 있으며, 초당 대량 저장이 필요하다. 따라서 **유연한 스키마와 높은 쓰기 성능을 가진 NoSQL DB** 가 적합하다.

서술형 2번. MLOps 도구를 활용한 'TO-BE' 인프라 구축

01. MLflow — 실험 추적 & 모델 레지스트리

모델을 학습할 때 MLflow Tracking으로 하이퍼파라미터, 학습 메트릭(Accuracy, F1 등), 데이터셋 버전을 자동 기록한다. 학습이 끝난 모델은 MLflow Model Registry에 등록하고, Staging → Production 단계를 거쳐 승인된 모델만 배포 파이프라인에 태운다. 이렇게 하면 '누가, 언제, 어떤 파라미터로 학습한 모델이 현재 서빙 중인지' 항상 추적할 수 있고, 문제 B 발생 시 이전 버전으로 빠르게 롤백할 수 있다.

02. Docker — 환경 격리 & 재현성

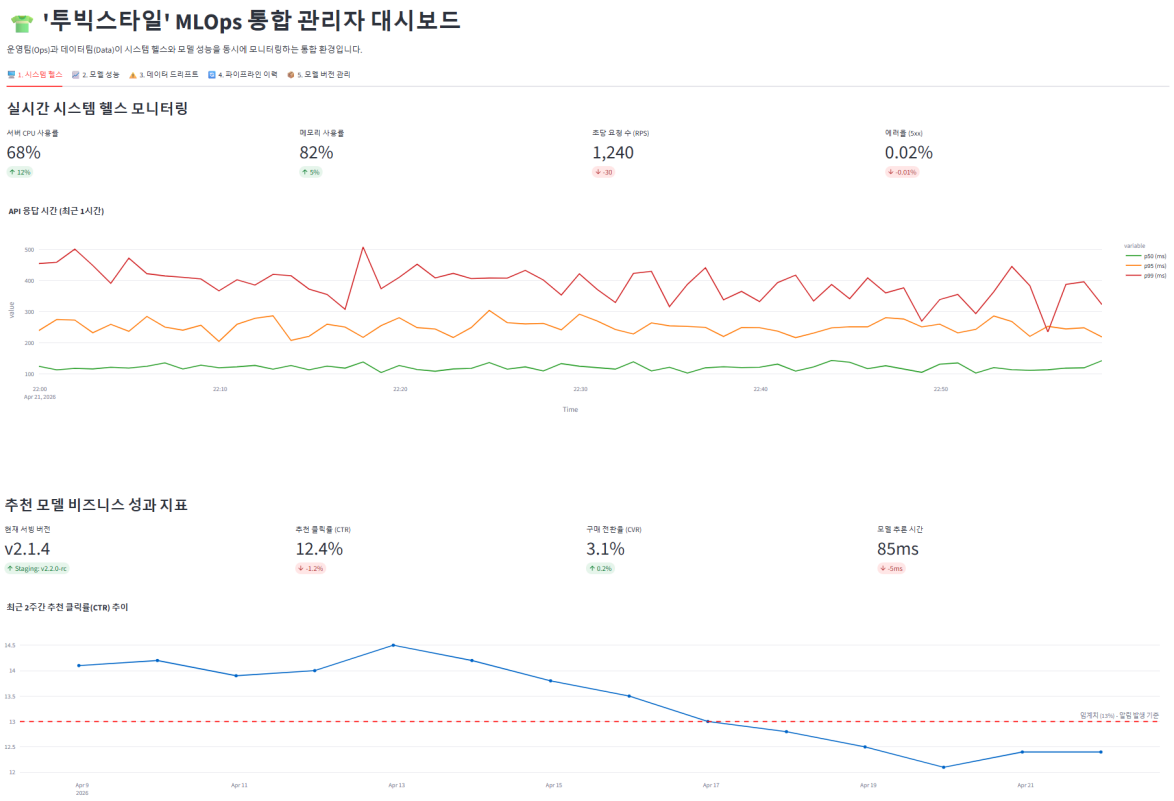
문제 A의 근본 원인은 개발 환경과 운영 환경의 차이다. 로컬에는 Python 3.9 + scikit-learn 1.2가 설치되어 있는데 서버에는 Python 3.11 + scikit-learn 1.4가 깔려 있다면 API 변경으로 예외가 발생할 수 있다. Docker를 도입하면 Dockerfile에 베이스 이미지, Python 버전, pip 패키지 버전을 고정하여 컨테이너 이미지를 빌드하므로, 어디서 실행하든 동일한 환경이 보장된다.

03. Apache Airflow — 파이프라인 오케스트레이션

모델 학습에는 데이터 수집 → 전처리 → 학습 → 평가 → 배포라는 일련의 단계가 있다. 이걸 수동으로 진행하면 단계 간 의존성 관리가 어렵고 실수가 발생하기 쉽다. Airflow를 사용하면 전체 워크플로를 DAG로 정의하고 스케줄링할 수 있다.

04. CT Pipeline — 지속적 학습

문제 B의 핵심은 모델 성능이 떨어져도 고객 불만 전까지 아무도 모른다는 것이다. CT Pipeline으로 두 가지 방식으로 해결한다.

1. 성능 모니터링 트리거를 설정한다. 서빙 중인 모델의 추천 클릭률(CTR)이나 전환율을 실시간 수집하고, 사전에 정한 임계치(예: CTR이 지난 주 대비 15% 이상 하락) 아래로 떨어지면 자동으로 재학습 파이프라인을 트리거한다.
2. 데이터 드리프트 감지를 적용한다. 패션 트렌드는 계절이나 유행에 따라 빠르게 변하므로, 학습 데이터와 실제 서비스 데이터의 분포가 달라질 수 있다. 입력 데이터의 통계적 분포 변화를 주기적으로 체크하여 드리프트가 감지되면 최신 데이터로 재학습을 자동화한다.

서술형 3번. 운영팀과 데이터팀이 함께 볼 수 있는 MLOps 관리자 대시보드 설계